L13: Principles of Learning

C. V. Jawahar

IIIT Hyderabad

February 19, 2026

Recap:

- Problems of interest:
  - Learn a function $y = f(\mathbf{W}, \mathbf{x})$ from the data.
    - (a) Classification (b) Regression
  - Learn Feature Transformations $\mathbf{x}' = \mathbf{W}\mathbf{x}$ or $\mathbf{x}' = f(\mathbf{W}, \mathbf{x})$
    - (a) Feature Normalization (b) PCA/Dimensionality Reduction
- Algorithms/Approaches:
  - (i) Nearest Neighbour Algorithm (ii) Linear Classification: $sign(\mathbf{w}^T\mathbf{x})$ (iii) Decide as ω_1 if $P(\omega_1|\mathbf{x}) \geq P(\omega_2|\mathbf{x})$ else ω_2.
  - Linear Regression: (a) closed form and (b) GD; Ridge and Lasso
  - Bayesian View; Normal Distribution; MLE and GMM
  - K Means; Naive Bayes and Logistic Regression
  - SVMs and Kernels
- Supervised Learning:
  - Notion of Training, Validation and Testing, Performance Metrics item How to practice (supervised) learning?
  - Low rank data matrix and SVD; PCA and Eigen Faces
  - Notion of Loss Function, (eg. MSE), Regularization.
  - Role of Optimization, Convex and non-Convex optimization
  - Closed form solution, Gradient Descent, Eigen vector solns, Expectation-Maximization (EM), MLE

SVM and Kernels: Recap

- Linear, hard and soft marghin
- Nonlinear Kernel SVM
- Training: What do we do in primal? dual?
- Testing: What do we do in primal? dual?
- ExOR

No Free Lunch and Ugly Duckling

No Free Lunch Core idea: No learning algorithm is universally best across all possible problems. **why?** If we average performance over every possible data generating process, every algorithm performs exactly the same. A method only looks strong when assumptions about the problem match the method bias.

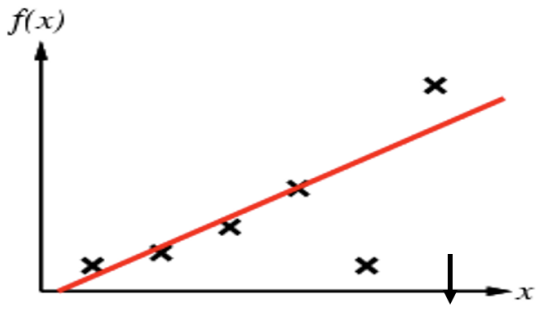
UIgly Ducking Core idea: Without assumptions or feature weighting, all objects are equally similar to each other. **Why?** If we count all possible properties that two objects may share, any pair shares the same number of properties as any other pair. Similarity only becomes meaningful when we choose which features matter.

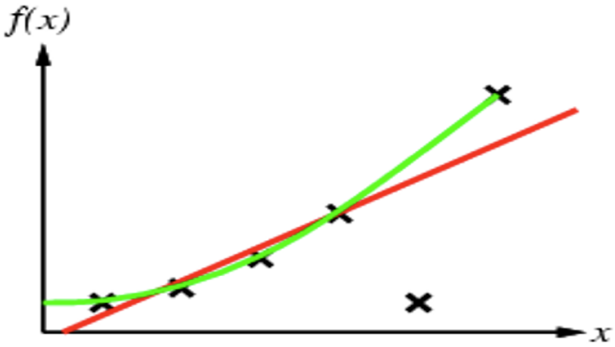
In short: No classifier is best for everything, and No representation/feature is best for everything.

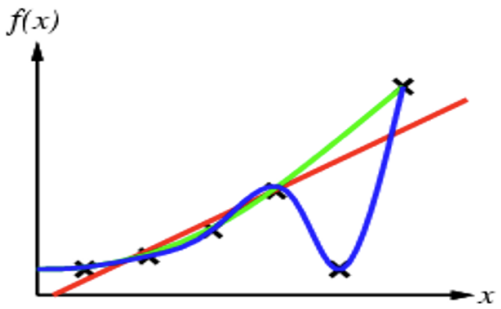
We do well in practice since, we design a solution for a specific problem/data.

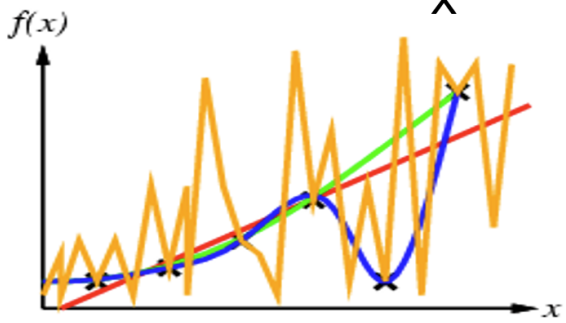
Regression: What is the right model?

Problem: We have 6 data samples (x, y). We want to predict y for a new x^*. which model is ideal?
Note: (i) red has error for all samples. (ii) green fits (passes through) all five, except one. (iii) blue fits all six perfectly. (iv) yellow also pass through all six.





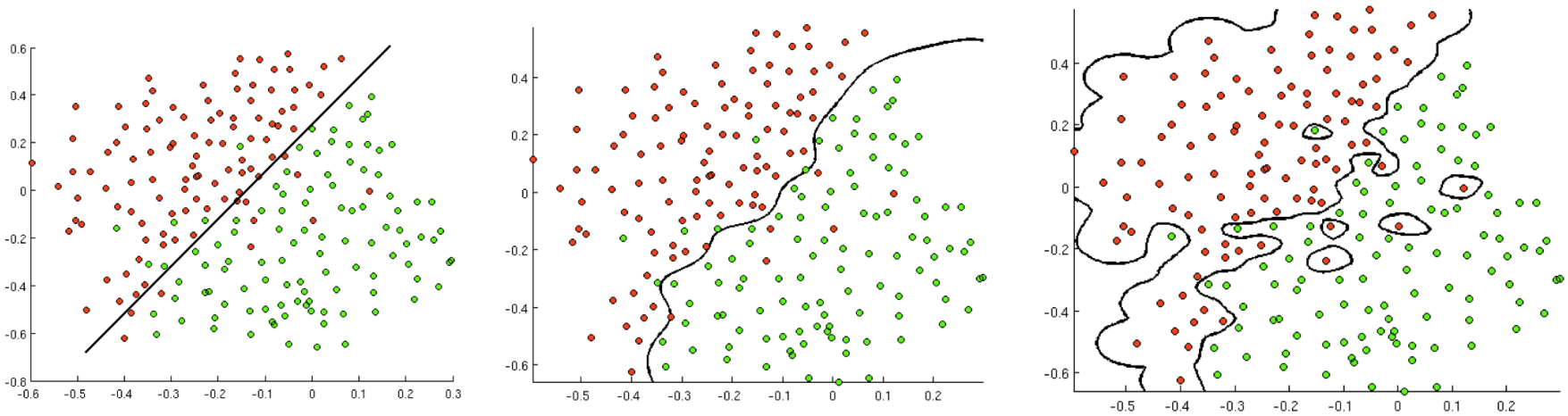




Given six "training" (x_i, y_i) pairs, find the y corresponding to the new "test" x*

Which curve is the best?

What is the right classifier/decision boundary?



What happens when we simply train?

- We often minimize an error over the training examples.
- The discrepancy between the actual and predicted is the loss $\mathcal{L}$. Let x_i is the input, z_i is the true output and y_i is the predicted output. $f()$ is the model, say a Neural Network with MSE loss

$$\mathcal{L} = \sum_i (z_i - y_i)^2 = \sum_i (z_i - f(x_i))^2$$

- If we want only to minimize this, we can even get zero error, with an appropriate function. (i.e, the perfect error/loss; do we want this?)
- If we do not want that. **Then what do we want?**

Two Wrong examples of finding a function from data

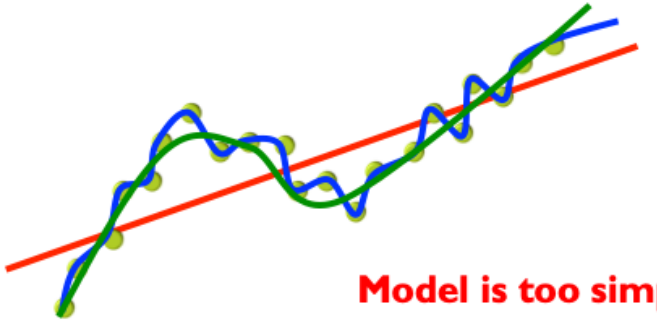
- Karnaugh Map (K-Map)
- Look up table (LUT)

They do not learn; **What do we mean by this?**

Generalization

- **Learning is concerned with accurate prediction of future data, not accurate prediction of training or available data.**
- (Occam's Razor) How?: Select the simplest hypothesis (solution) that suits the data. Eg. Minimize Sum of "fit error" and "degree of the polynomial"
- (Do you see how our regularization also helps in this?)

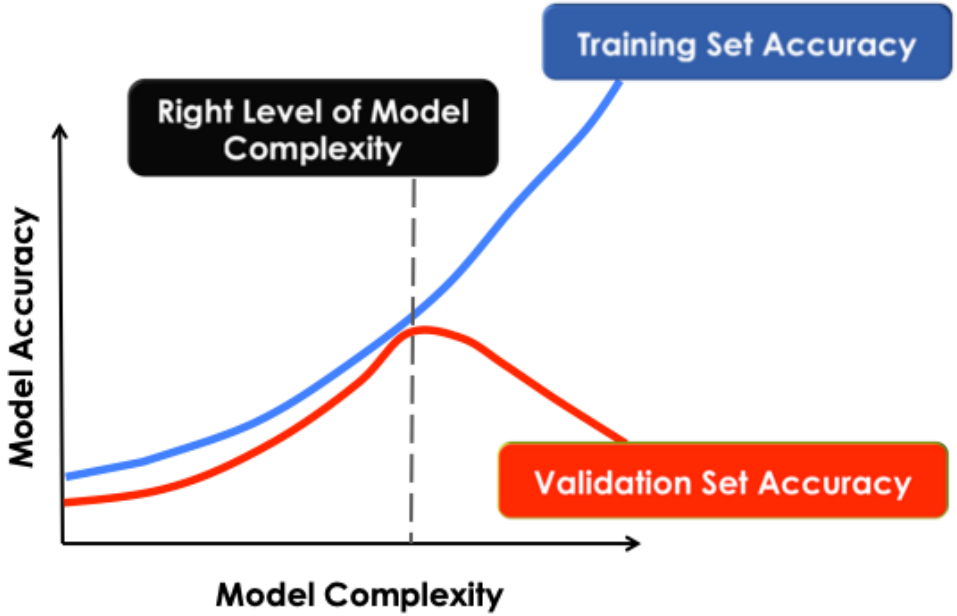
Generalize; Don't Memorize or Overfit



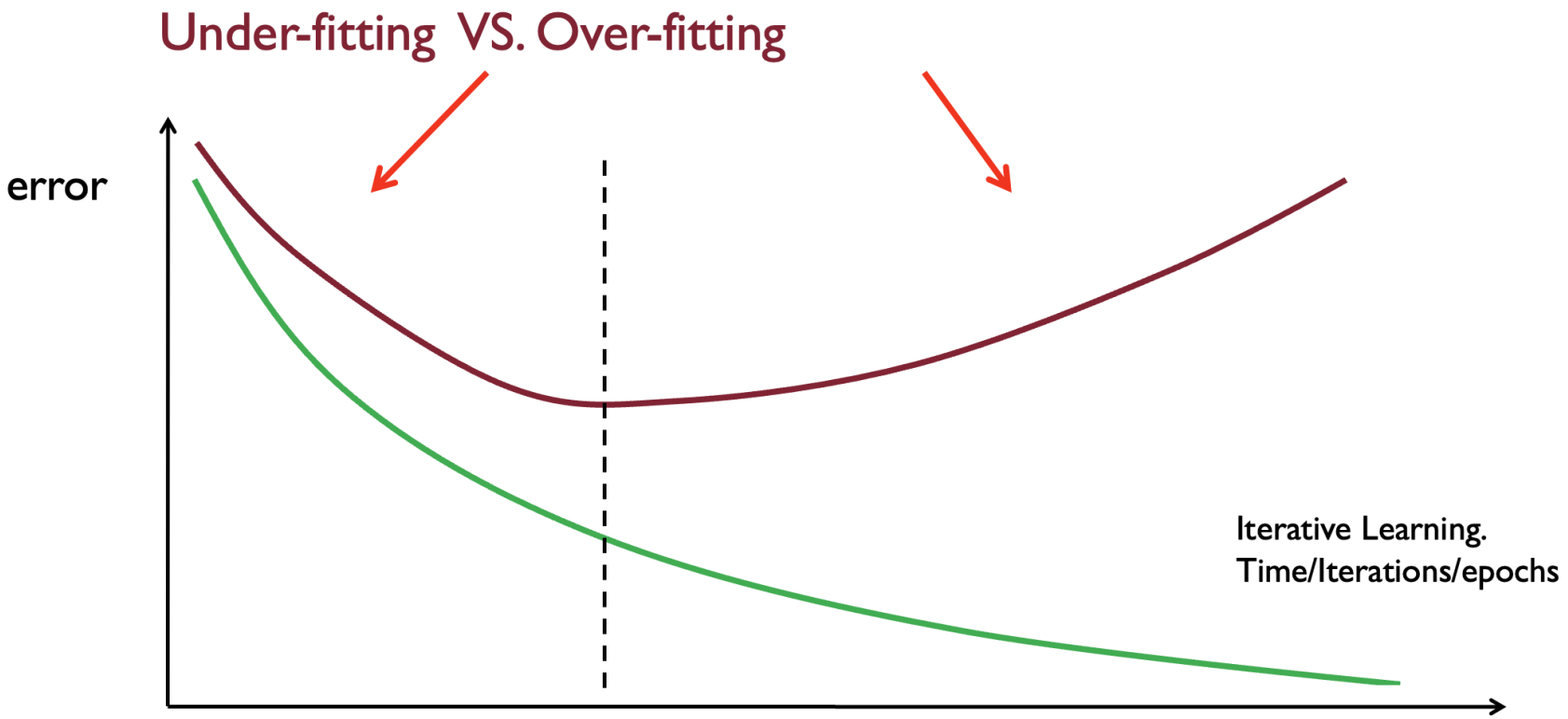
Model is too simple → UNDER LEARN

Model is too complex → MEMORIZE

Model is just right → GENERALIZE



Fit Error for iterative training



Overfitting

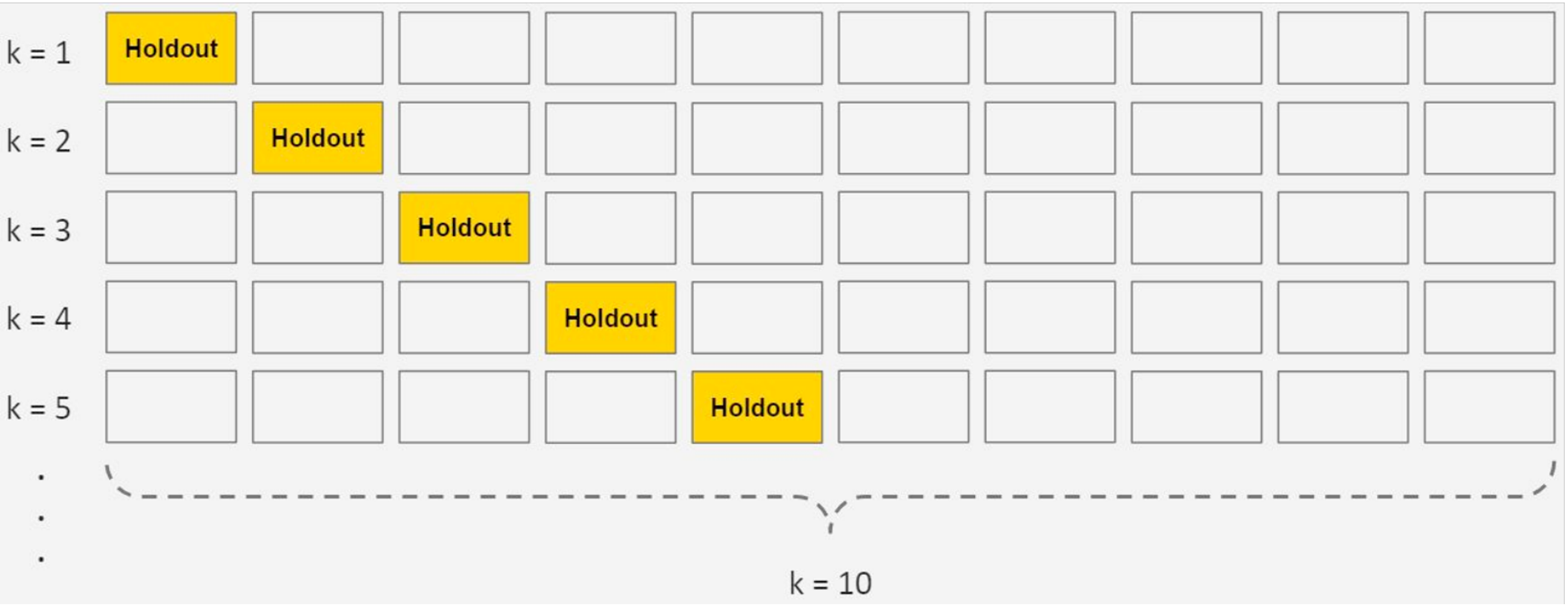
Overfitting is a modeling error which occurs when a function is too closely fit to a limited set of data points. Overfitting the model generally takes the form of making an overly complex model to explain idiosyncrasies in the data under study. In reality, the data often studied has some degree of error or random noise within it. Thus attempting to make the model conform too closely to slightly inaccurate data can infect the model with substantial errors and reduce its predictive power

- Overfit if $TrainError << Test/DeploymentError$

How to Avoid Overfitting?

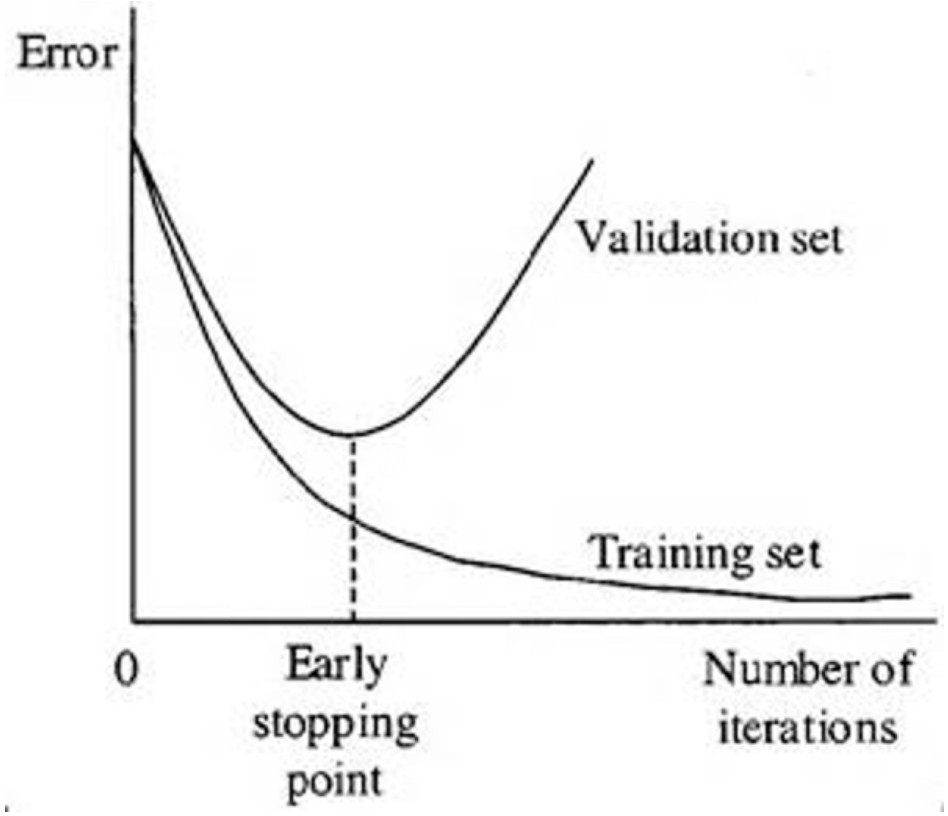
- Cross Validation
- Train with more data
- Reduce/Remove features
- Early Stopping
- Regularization
- Ensembling

Cross Validate



- Split the training data into k subsets. Use k-1 for training and one for evaluation. Final error is the average of all the k sets. (k-fold cross validation)
- Split the data into x and 100-x percentages for train and evaluation. Repeat the task many times and average the error.
- If number of samples (N) are small, use N-1 for training and one for testing. Repeat N times and average. Leave One Out (LOO) error.

Early Stop in Iterative Training



Regularization

Regularization is a process of introducing additional information in order to solve an ill-posed problem or to prevent overfitting.

- We were minimizing the fit error

$$\mathcal{L} = \sum_i (z_i - y_i)^2 = \sum_i (z_i - f(x_i))^2$$

- We can minimize
  - Fit Error ($\mathcal{L}$) + some measure of complexity of the model
  - $\mathcal{L}$ + Sum of Weights
  - $\mathcal{L}$ + Number of Nonzero Weights
- Data Split:
  - **Train:** Data used for training; or find learnable parameters of the model
  - **Validation:** "internal" test data (Estimate the error on one or more subsets (eg. Crossvalidation) Vary hyper-parameters (eg. Architecture of a Neural Network) and retrain
  - **Test**: Evaluate the final (best) model on test data.

Bias and Variance

- **Bias** Error from overly simple assumptions. In this case, model cannot represent true pattern. Eg. Fitting a straight line to quadratic data.
- **Variance** Error from sensitivity to training data. i.e., Model changes a lot if dataset changes slightly. Eg. Fitting a straight line to quadratic data. or Deep tree that memorizes noise.
- Expected prediction error = Error from bias + Error from variance + Erreducible noise
- Bias is error from wrong assumptions. Variance is error from sensitivity to data. Learning is choosing the right complexity.

Bias and Variance

- **Linear regression:** High bias if data nonlinear; Low variance
- **Polynomial regression** Bias decreases as degree increases; Variance increases
- **KNN** Small k leads to low bias high variance; and Large k leadas to high bias low variance
- **Decision trees** Shallow trees leads to high bias; and Deep trees leads to high variance
- **Neural networks** Small network leads to high bias, and Huge network leads to high variance unless regularized

Parameters and Hyperparameters

Parameters: These are values learned directly from data during training. The algorithm adjusts them automatically to minimize loss or maximize likelihood.
Examples (i) Linear regression weights and bias (ii) Logistic regression weight vector (iii) Neural network weights (iv) Gaussian mean and variance (v) Decision tree split thresholds

Hyperparameters: These are settings chosen before training. They control how the model learns rather than what it learns.
Examples (i) Learning rate (ii) Number of clusters in K means (iii) MaxAllowedDepth of decision tree (iv) Regularization strength lambda (v) Number of hidden layers

Discussions Point

In supervised learning, we use the training data $\mathcal{D}_{Tr}$ to develop a solution and then evaluate it on the test data $\mathcal{D}_{Te}$

Q: Why is that a solution optimized only on training data as

$$\min_{\mathbf{W},} \mathcal{L}(\mathbf{W}, \mathbf{x}) \quad \mathbf{x} \in \mathcal{D}_{Tr}$$

works reasonably well on the test data also?

1. We believe in luck.
2. Once it is trained on $\mathcal{D}_{Tr}$, it then works well on <u>any</u> data.
3. Actually, we use $\mathcal{D}_{Te}$ <u>also</u> for training. Eqn. is incorrect.
4. What we care is actually the performance on the $\mathcal{D}_{Tr}$; not $\mathcal{D}_{Te}$.
5. It is some magic. No explanation found till today.

Discussion Point

Consider a GDbased Training of NN from random initialization, with three discrete configurations (like number of layers of a NN). Suggest how do we find the best model in a principled manner?
How do we write? (write pseudo code)

- Initialization
- Number of Layers
- Cross validation

How do we get the final best solution?
(Post lecture do this in detail)